

Bplus Tree

B+ 树

引入

B+ 树是 [B 树](#) 的一个升级，它比 B 树更适合实际应用中操作系统的文件索引和数据库索引。目前现代关系型数据库最广泛的支持索引结构就是 B+ 树。

B+ 树是一种多叉排序树，即每个节点通常有多个孩子。一棵 B+ 树包含根节点、内部节点和叶子节点。根节点可能是一个叶子节点，也可能是一个包含两个或两个以上孩子节点的节点。

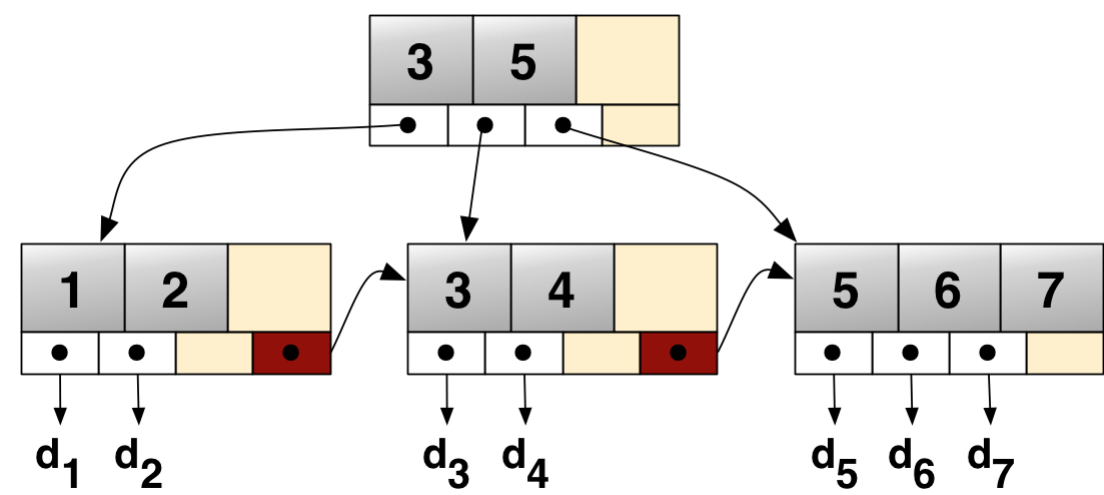
B+ 树的特点是能够保持数据稳定有序，其插入与修改拥有较稳定的对数时间复杂度。B+ 树元素自底向上插入，这与二叉树恰好相反。

首先介绍一棵 阶 B+ 树的特性。表示这个树的每一个节点最多可以拥有的子节点个数。一棵 阶的 B+ 树和 B 树的差异在于：

1. 有 棵子树的节点中含有 个关键字（即将区间分为 个子区间，每个子区间对应一棵子树）。
2. 所有叶子节点中包含了全部关键字的信息，及指向含这些关键字记录的指针，且叶子节点本身依关键字的大小自小到大顺序链接。
3. 所有的非叶子节点可以看成是索引部分，节点中仅含有其子树（根节点）中的最大（或最小）关键字。
4. 除根节点外，其他所有节点中所含关键字的个数最少有 （注意：B 树中除根以外的所有非叶子节点至少有 棵子树）。

同时，B+ 树为了方便范围查询，叶子节点之间还用指针串联起来。

以下是一棵 B+ 树的典型结构：



B+ 树相比于 B 树的优势

由于索引节点上只有索引而没有数据，所以索引节点上能存储比 B 树更多的索引，这样树的高度就会更矮。树的高度越矮，磁盘寻道的次数就会越少。

因为数据都集中在叶子节点，而所有叶子节点的高度相同，那么可以在叶子节点中增加前后指针，指向同一个父节点的相邻兄弟节点，这样可以更好地支持查询一个值的前驱或后继，使连续访问更容易实现。

比如这样的 SQL 语句：select * from tbl where t > 10，如果使用 B+ 树存储数据的话，可以首先定位到数据为 10 的节点，再沿着它的 next 指针一路找到所有在该叶子节点右边的叶子节点，返回这些节点包含的数据。

而如果使用 B 树结构，由于数据既可以存储在内部节点也可以存储在叶子节点，连续访问的实现会更加繁琐（需要在树的内部结构中进行移动）。

过程

与 [B 树](#) 类似，B+ 树的基本操作有查找，遍历，插入，删除。

查找

B+ 树的查找过程和 B 树类似。假设需要查找的键值是，那么从根节点开始，从上到下递归地遍历树。在每一层上，搜索的范围被减小到包含搜索值的子树中。

一个实例：在如下这棵 B+ 树上查找 45。

- 先和根节点比较
- 因为根节点的键值比 45 要小，所以去往根节点的右子树查找
- 因为 45 比 35 大，所以要与右边的索引相比
- 右侧的索引也为 45，所以要去往该节点的右子树继续查找
- 然后就可以找到 45

需要注意的是，在查找时，若非叶子节点上的关键字等于给定值，并不终止，而是继续向下直到叶子节点。因此，在 B+ 树中，不管查找成功与否，每次查找都是走了一条从根到叶子节点的路径。其余同 B 树的查找类似。

查找一个键的代码如下：

▼ 实现

```
1 T find(V key) {
2   int i = 0;
3   while (i < this.number) {
4     if (key.compareTo((V) this.keys[i]) <= 0) break;
5     i++;
6   }
7   if (this.number == i) return null;
8   return this.childs[i].find(key);
9 }
```

遍历

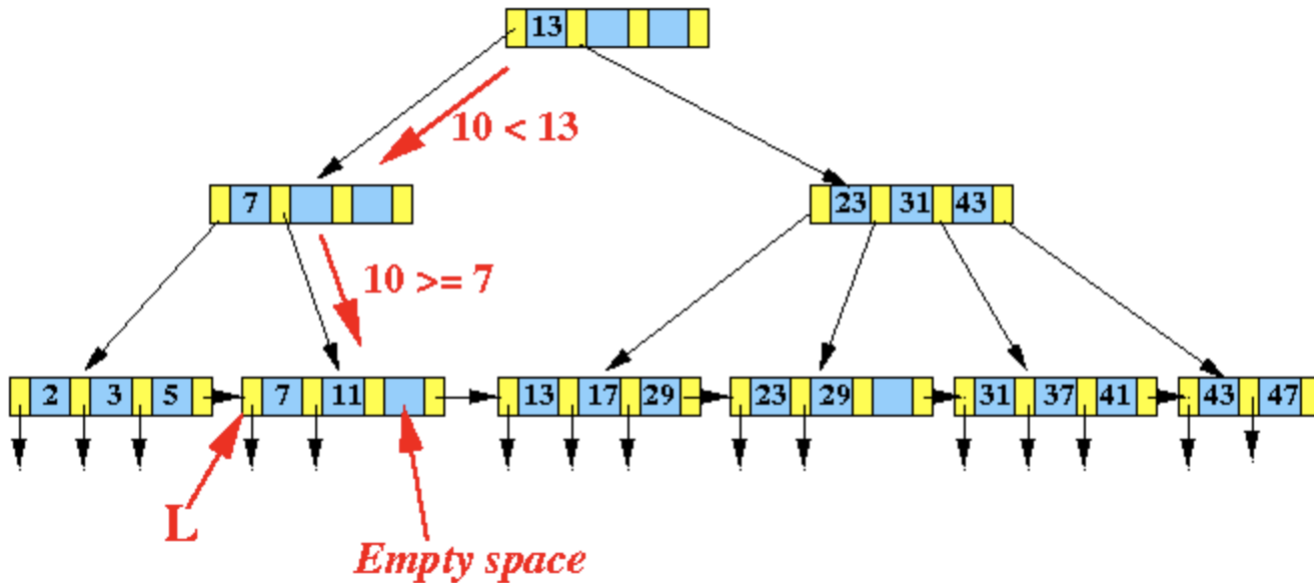
B+ 树只在叶子节点的层级上就可以实现整棵树的遍历。从根节点出发一路搜索到最左端的叶子节点之后即可根据指针遍历。

插入

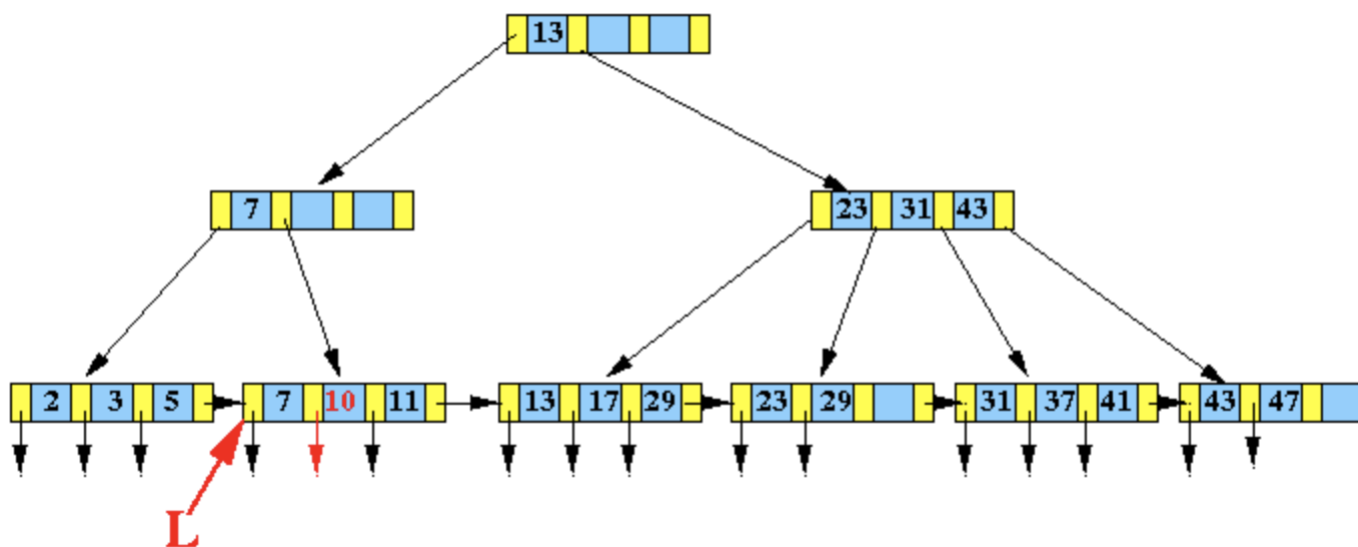
B+ 树的插入算法与 B 树的相近：

- 1. 若为空树，创建一个叶子节点，然后将记录插入其中，此时这个叶子节点也是根节点，插入操作结束。
- 2. 针对叶子类型节点：根据关键字找到叶子节点，向这个叶子节点插入记录。插入后，若当前节点关键字的个数小于 $m-1$ ，则插入结束。否则将这个叶子节点分裂成左右两个叶子节点，左叶子节点包含前 $\lfloor \frac{m-1}{2} \rfloor$ 个记录，右节点包含剩下的记录，将第 $\lfloor \frac{m-1}{2} \rfloor + 1$ 个记录的关键字进位到父节点中（父节点一定是索引类型节点），进位到父节点的关键字左孩子指针向左节点，右孩子指针向右节点。将当前节点的指针指向父节点，然后执行第 3 步。
- 3. 针对索引类型节点（内部节点）：若当前节点关键字的个数小于等于 $m-2$ ，则插入结束。否则，将这个索引类型节点分裂成两个索引节点，左索引节点包含前 $\lfloor \frac{m-2}{2} \rfloor$ 个 key，右节点包含 $\lfloor \frac{m-2}{2} \rfloor + 1$ 个 key，将第 $\lfloor \frac{m-2}{2} \rfloor + 1$ 个关键字进位到父节点中，进位到父节点的关键字左孩子指向左节点，进位到父节点的关键字右孩子指向右节点。将当前节点的指针指向父节点，然后重复这一步。

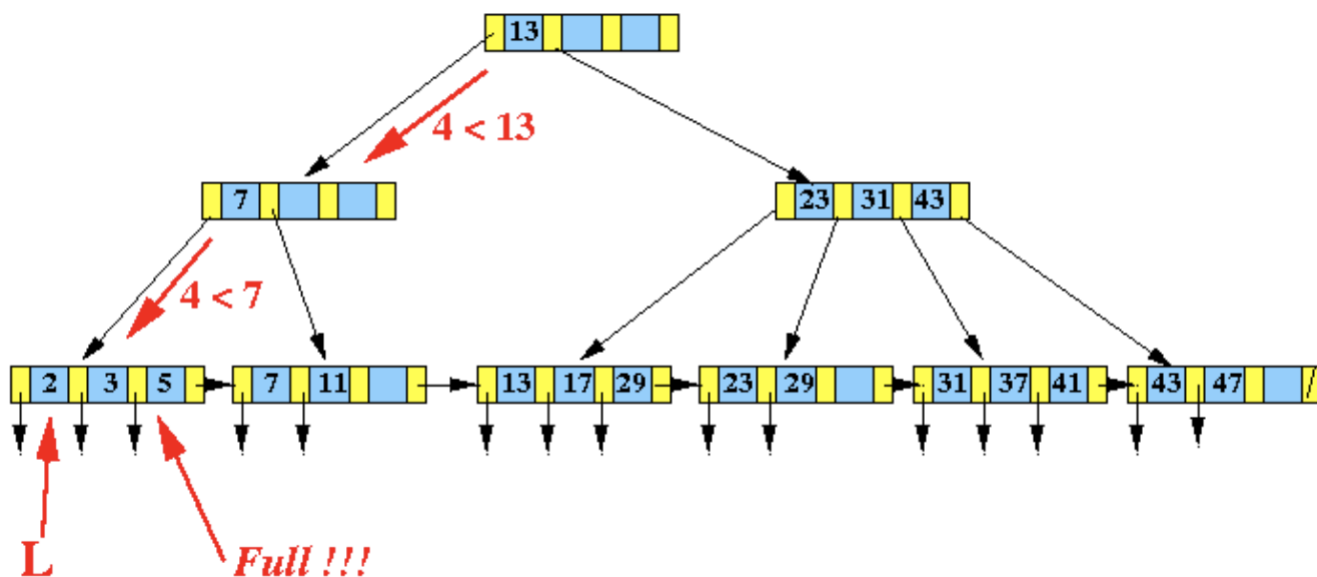
比如在下图的 B+ 树中，插入新的数据 10：



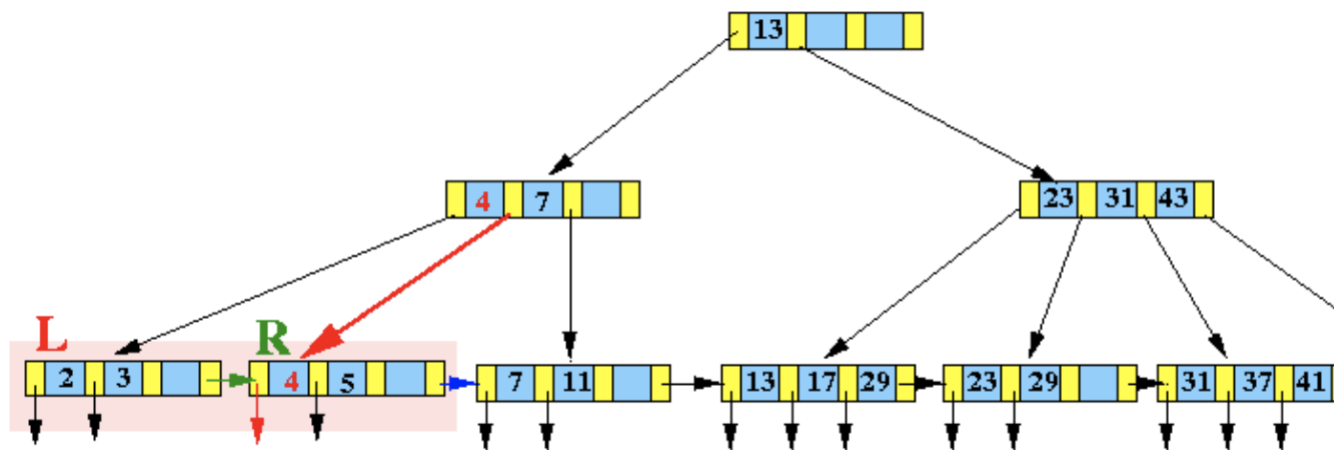
由于插入节点 在插入之后并没有溢出，所以可以直接变成 。



而如下图的 B+ 树中，插入数据 4:



由于所在节点 在插入之后数据溢出，因此需要分裂为两个新的节点，同时调整父节点的索引数据：



分裂成了 和 , 因此需要在这两个节点之间新增一个索引值, 这个值应该满足:

1. 大于左子树的最大值;
2. 小于等于右子树的最小值。

综上, 需要在父节点中新增索引 4 和两个指向新节点的指针。

更多的例子可以参考演示网站 [BPlustree](#)

插入一个键的代码如下:

▼ 实现

```
1 void BPTree::insert(int x) {
2     if (root == NULL) {
3         root = new Node;
4         root->key[0] = x;
5         root->IS_LEAF = true;
6         root->size = 1;
7         root->parent = NULL;
8     } else {
9         Node* cursor = root;
10        Node* parent;
11
12        while (cursor->IS_LEAF == false) {
13            parent = cursor;
14            for (int i = 0; i < cursor->size; i++) {
15                if (x < cursor->key[i]) {
16                    cursor = cursor->ptr[i];
17                    break;
18                }
19
20                if (i == cursor->size - 1) {
21                    cursor = cursor->ptr[i + 1];
22                    break;
23                }
24            }
25        }
26        if (cursor->size < MAX) {
27            insertVal(x, cursor);
28            cursor->parent = parent;
29            cursor->ptr[cursor->size] = cursor->ptr[cursor->size - 1];
30            cursor->ptr[cursor->size - 1] = NULL;
31        } else
32            split(x, parent, cursor);
33    }
34 }
35
36 void BPTree::split(int x, Node* parent, Node* cursor) {
37     Node* LLeaf = new Node;
38     Node* RLeaf = new Node;
39     insertVal(x, cursor);
40     LLeaf->IS_LEAF = RLeaf->IS_LEAF = true;
41     LLeaf->size = (MAX + 1) / 2;
42     RLeaf->size = (MAX + 1) - (MAX + 1) / 2;
43     for (int i = 0; i < MAX + 1; i++) LLeaf->ptr[i] = cursor->ptr[i];
44     LLeaf->ptr[LLeaf->size] = RLeaf;
45     RLeaf->ptr[RLeaf->size] = LLeaf->ptr[MAX];
46     LLeaf->ptr[MAX] = NULL;
47     for (int i = 0; i < LLeaf->size; i++) {
48         LLeaf->key[i] = cursor->key[i];
49     }
50     for (int i = 0, j = LLeaf->size; i < RLeaf->size; i++, j++) {
51         RLeaf->key[i] = cursor->key[j];
52     }
53     if (cursor == root) {
54         Node* newRoot = new Node;
55         newRoot->key[0] = RLeaf->key[0];
56         newRoot->ptr[0] = LLeaf;
57         newRoot->ptr[1] = RLeaf;
58         newRoot->IS_LEAF = false;
59         newRoot->size = 1;
60         root = newRoot;
61         LLeaf->parent = RLeaf->parent = newRoot;
62     } else {
63         insertInternal(RLeaf->key[0], parent, LLeaf, RLeaf);
64     }
65 }
66
67 void BPTree::insertInternal(int x, Node* cursor, Node* LLeaf, Node* RRLeaf) {
68     if (cursor->size < MAX) {
69         auto i = insertVal(x, cursor);
70         for (int j = cursor->size; j > i + 1; j--) {
71             cursor->ptr[j] = cursor->ptr[j - 1];
72         }
73         cursor->ptr[i] = LLeaf;
74         cursor->ptr[i + 1] = RRLeaf;
75     }
76
77     else {
78         Node* newLchild = new Node;
79         Node* newRchild = new Node;
80         Node* virtualPtr[MAX + 2];
81         for (int i = 0; i < MAX + 1; i++) {
82             virtualPtr[i] = cursor->ptr[i];
83         }
84         int i = insertVal(x, cursor);
85         for (int j = MAX + 2; j > i + 1; j--) {
86             virtualPtr[j] = virtualPtr[j - 1];
87         }
88         virtualPtr[i] = LLeaf;
89         virtualPtr[i + 1] = RRLeaf;
90         newLchild->IS_LEAF = newRchild->IS_LEAF = false;
91         // 这里和叶子节点有区别
```

```
92     newLchild->size = (MAX + 1) / 2;
93     newRchild->size = MAX - (MAX + 1) / 2;
94     for (int i = 0; i < newLchild->size; i++) {
95         newLchild->key[i] = cursor->key[i];
96     }
97     for (int i = 0, j = newLchild->size + 1; i < newRchild->size; i++, j++) {
98         newRchild->key[i] = cursor->key[j];
99     }
100    for (int i = 0; i < LLeaf->size + 1; i++) {
101        newLchild->ptr[i] = virtualPtr[i];
102    }
103    for (int i = 0, j = LLeaf->size + 1; i < RLeaf->size + 1; i++, j++) {
104        newRchild->ptr[i] = virtualPtr[j];
105    }
106    if (cursor == root) {
107        Node* newRoot = new Node;
108        newRoot->key[0] = cursor->key[newLchild->size];
109        newRoot->ptr[0] = newLchild;
110        newRoot->ptr[1] = newRchild;
111        newRoot->IS_LEAF = false;
112        newRoot->size = 1;
113        root = newRoot;
114        newLchild->parent = newRchild->parent = newRoot;
115    } else {
116        insertInternal(cursor->key[newLchild->size], cursor->parent, newLchild,
117                      newRchild);
118    }
119 }
120 }
```

删除

B+ 树的删除也仅在叶子节点中进行，当叶子节点中的最大关键字被删除时，其在非叶子节点中的值可以作为一个分界关键字存在。若因删除而使节点中关键字的个数少于 $\frac{1}{2}$ 时，其和兄弟节点的合并过程亦和 B 树类似。

具体步骤如下：

1. 首先查询到键值所在的叶子节点，删除该叶子节点的数据。
2. 如果删除叶子节点之后的数据数量，满足 B+ 树的平衡条件，则直接返回。
3. 否则，就需要做平衡操作：如果该叶子节点的左右兄弟节点的数据量可以借用，就借用过来满足平衡条件。否则，就与相邻的兄弟节点合并成一个新的子节点了。

在上面平衡操作中，如果是进行了合并操作，就需要向上修正父节点的指针：删除被合并节点的键值以及指针。

由于做了删除操作，可能父节点也会不平衡，那么就按照前面的步骤也对父节点进行重新平衡操作，这样一直到某个节点平衡为止。

可以参考 [B 树](#) 中的删除章节。

► 实现

参考资料

- [B+ tree wikipedia](#)
- [B 树、B+ 树索引算法原理（下）](#)
- [B+ 树详解 + 代码实现（插入篇）](#)
- [Deletion from a B+ Tree](#)

本页面最近更新：2025/3/20 15:39:08，[更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Persdre](#), [aofall](#), [CainXCain](#), [CoelacanthusHex](#), [HeRaNO](#), [iamtwz](#), [ja-king-zh](#), [LTHAndy](#), [Marcythm](#), [shuzhouliu](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用